

Notebook

February 8, 2026

1 Атака на оракул по младшему биту RSA

Представьте, что у Вас есть побочный канал на сервере, принимающем шифротексты RSA, который позволяет Вам получить только младший бит соответствующего открытого текста. Кажется, что не получится забрать с сервера достаточно полезной информации, так? На самом деле нет. Даже утечки в один бит достаточно, чтобы полностью сломать криптосистему. Вот, как это сделать. Вспомним, что RSA - это гомоморфное шифрование, и значит:

$$\forall M_1, M_2 \in Z_N^*, C_1 C_2 \bmod N = (M_1 M_2)^e \bmod N = M_1^e M_2^e \bmod N$$

Отдельный случай:

$$Enc(2M) = 2^e M^e \bmod N$$

Мы можем воспользоваться этим свойством для получения открытого текста. Давайте посмотрим, какой младший бит мы получим, если умножим открытый текст M на 2. Можно подумать, что он всегда будет 0, раз младший бит целого числа, умноженного на 2 - 0. Но мы умножаем по модулю N , и если получившийся открытый текст больше или равен N , мы вычитаем из него N , пока он не станет меньше, чем N . Что конкретно происходит, если мы умножим открытый текст на 2? Мы знаем, что M изначально меньше, чем N :

$$M < N$$

Значит

$$2M < 2N$$

и есть два случая:

$$2M < N$$

и

$$2M > N$$

$2M \neq N$ так как N - нечётное. В случае когда $2M < N$ мы не вычитаем N из результата умножения, и $LSB(2M \bmod N) = 0$. Однако если $2M > N$, то получившийся открытый текст - это $2M - N$, и раз N - нечётное, $LSB(2M \bmod N) = 1$. Таким образом мы можем определить, что $2M < N$ или что $2M > N$, что значит $M < \lceil \frac{N}{2} \rceil$ или $M \geq \lceil \frac{N}{2} \rceil$. Мы можем разделить область значений M пополам. И мы можем продолжить этот процесс. Если $LSB(4M) = 0$, то M находится в левой половине ранее поделенной пополам области значений, а если $LSB(4M) = 1$, то он находится в правой половине. Мы можем продолжать этот процесс пока либо не

узнаем M или область значений M будет настолько маленькой, что легче будет проверить все значения, чем запрашивать биты с сервера.

Вам предоставлен сервер, который представляет собой Оракул по младшему биту. Вы можете отправлять закрытые тексты и он вернёт младший бит открытого текста. Восстановите изначальный открытый текст, используя оракул. Удачи!

```
[ ]: import socket
import re
from Crypto.Util.number import inverse, long_to_bytes, bytes_to_long
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone', 1343))
        if show:
            print (self.recv_until().decode())
    def recv_until(self, symb=b'\n>'):
        """Получаем сообщения с сервера, по дефолту до знака_
↪приглашения"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_key(self, show=True):
        """Получаем открытый ключ с сервера"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        e=int(re.search('(?<=e: )\d+', response).group(0))
        N=int(re.search('(?<=N: )\d+', response).group(0))
        return (e,N)
    def get_ciphertext(self, show=True):
        """Получаем шифротекст с сервера"""
        self.s.sendall('ciphertext\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        c=bytes_to_long(bytes.fromhex(re.search('(?<=ciphertext: )[0-9a-f]+', response).group(0)))
        return c
```

```

def get_LSB_from_ciphertext(self, ciphertext, show=True):
    """Получаем младший бит открытого текста для соответствующего,
    ↪шифротекста"""
    if isinstance(ciphertext,int):
        ciphertext=long_to_bytes(ciphertext)
    if len(ciphertext)>256:
        print ('Ciphertext too long')
        return None
    if len (ciphertext)<256:
        ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
    ciphertext_hex=ciphertext.hex()

    self.s.sendall(f'lsb {ciphertext_hex}\n'.encode())

    response=self.recv_until().decode()
    if show:
        print (response)
    lsb=int(re.search(r'(?<=lsb is: )\d',response).group(0))

    return lsb

def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
      (e,N)=vs.get_public_key()
      c=vs.get_ciphertext()
      print(c)
      print (vs.get_LSB_from_ciphertext(c, False))
      # Советую использовать опцию show=False. 2000 запросов с полным,
      ↪Выводом - СЛИШКОМ МНОГО.

```

```

[ ]:

```